Ciência de Dados para Negócios
Aprendizagem de Máquina - II

Prof. Luis Claudio

ATENÇÃO:

Escrever somente os nomes dos participantes presentes em sala no dia.

Após o final da atividade, o grupo se desfaz (não precisa, necessariamente, manter os mesmos alunos no mesmo grupo para a próxima vez).

Integrantes	**30/09/2026**
1. **Débora Kocks Nogueira**	
2. **Gabriel Yuji Shimoda**	
3. **Stefanie Mayumi Inacio Kobayashi**	
4.	
5.	
6.	

Obs.:
- Trabalho presencial. Não coloque nomes de quem não está em sala da aula.
- Mínimo 03 alunos; máximo de 06 alunos.

Situação-Problema

Uma empresa de comércio eletrônico possui milhares de produtos cadastrados em sua plataforma. Durante o cadastro de um novo produto, o vendedor precisa selecionar manualmente sua categoria, como camiseta, calçado, bolsa ou acessório.

A empresa percebeu que erros nessa classificação são frequentes. Produtos cadastrados em categorias incorretas prejudicam pesquisas realizadas pelos clientes, filtros de navegação e algumas análises comerciais.

A equipe de Ciência de Dados recebeu, portanto, a tarefa de estudar uma solução capaz de classificar automaticamente a categoria de um produto a partir de sua imagem.

Para um primeiro experimento controlado, foi preparado um conjunto de treinamento contendo 12.800 imagens distribuídas entre quatro categorias: camiseta, calçado, bolsa e acessório.

Todas as imagens possuem dimensão 32 × 32 pixels e utilizam o padrão RGB.

A equipe decidiu inicialmente construir uma MLP (Multilayer Perceptron), arquitetura já utilizada em outros projetos da empresa.

Como uma MLP recebe seus dados na forma de valores de entrada, cada imagem será transformada em um vetor. Uma imagem RGB de 32 × 32 possui 32 × 32 × 3 = 3.072 valores de entrada.

Arquitetura proposta para a MLP

Parte da rede	Configuração	Ativação
Entrada	3.072 valores	—
1ª camada oculta	32 neurônios	ReLU
2ª camada oculta	8 neurônios	ReLU
Camada de saída	4 neurônios	Softmax

Configuração inicial de treinamento

Hiperparâmetro	Valor
batch_size	128
epochs	25
loss function	Categorical Cross-Entropy
optimizer	Adam
learning rate	0,001

Durante a discussão do projeto, um dos integrantes da equipe chamou atenção para o fato de que, na MLP, cada neurônio da primeira camada oculta estará conectado aos 3.072 valores provenientes da imagem. Outro integrante sugeriu então avaliar uma Rede Neural Convolucional (CNN), arquitetura desenvolvida especialmente para dados com estrutura espacial, como imagens.

Para iniciar essa análise, foi proposta uma primeira camada convolucional:

```
Conv2D(
    filters=12,
    kernel_size=(3,3),
    activation='relu'
)
```

Essa camada receberá diretamente as imagens RGB.

O objetivo da equipe, neste momento, não é determinar qual das duas arquiteturas apresentará melhor desempenho. Antes disso, é necessário compreender como cada uma processa as informações, como seus parâmetros são organizados e como ocorre seu treinamento.

Questões

1. Parâmetros e hiperparâmetros

Considerando a MLP descrita, explique a diferença entre parâmetros e hiperparâmetros.

Parâmetros são valores aprendidos pela própria rede durante o treinamento (pesos e biases), ajustados a cada lote para reduzir a perda. Hiperparâmetros são configurações definidas pelo analista antes do treinamento, ligadas à arquitetura ou ao processo de treinamento, e não são aprendidos pela rede. Nesta MLP, os 98.636 pesos e biases são parâmetros; camadas, neurônios, ativações, batch_size, épocas, função de perda, otimizador e learning rate são hiperparâmetros.

Em seguida, identifique no projeto:

a) quais elementos serão aprendidos pela rede durante o treinamento;

Os pesos de todas as conexões e os biases de cada neurônio das camadas ocultas e de saída. Eles são ajustados pelo otimizador a partir dos gradientes.

b) quais elementos correspondem a hiperparâmetros de arquitetura;

Quantidade de camadas ocultas (2); quantidade de neurônios por camada (32, 8 e 4 na saída); funções de ativação (ReLU nas ocultas e Softmax na saída); quantidade de valores de entrada (3072, definida pela imagem 32×32×3).

c) quais correspondem a hiperparâmetros de treinamento.

batch_size = 128; epochs = 25; função de perda = Categorical Cross-Entropy; otimizador = Adam; learning rate = 0,001.

2. Parâmetros treináveis da MLP

Calcule a quantidade de parâmetros treináveis em cada parte da MLP apresentada. Considere pesos e biases.

Camada de entrada: 0 (apenas recebe e repassa os 3072 valores)

1ª camada oculta (3.072 → 32): 3072 × 32 + 32 = 98304 pesos + 32 biases = 98336

2ª camada oculta (32 → 8): 32 × 8 + 8 = 256 pesos + 8 biases = 264

Camada de saída (8 → 4): 8 × 4 + 4 = 32 pesos + 4 biases = 36

Total de parâmetros treináveis: 98336 + 264 + 36 = 98636 parâmetros treináveis

3. Camada de entrada

Embora a imagem produza 3.072 valores de entrada, a camada de entrada da MLP não acrescenta parâmetros treináveis. Explique:
a) por que essa camada possui zero parâmetros treináveis;

Porque ela apenas recebe os 3072 valores da imagem e os repassa à primeira camada oculta. Não realiza soma ponderada nem aplica ativação, logo não possui pesos nem biases próprios.

b) por que, mesmo assim, a dimensionalidade da entrada influencia diretamente a quantidade de parâmetros da primeira camada oculta.

Porque a MLP é densa. Cada um dos 32 neurônios da primeira camada oculta se conecta a todos os 3072 valores de entrada, com um peso para cada conexão. Assim, a quantidade de entradas determina diretamente o número de pesos: 3072 × 32 = 98304, além dos 32 biases.

4. Batch size e épocas

O conjunto de treinamento possui 12.800 imagens e será utilizado com batch_size = 128 e epochs = 25. Determine:
a) quantos lotes serão processados durante uma época;

12800 ÷ 128 = 100 lotes por época.

b) quantas atualizações dos parâmetros serão realizadas durante uma época;

Cada lote gera uma atualização dos parâmetros. Portanto, 100 atualizações por época.

c) quantas atualizações serão realizadas durante todo o treinamento.

100 atualizações por época × 25 épocas = 2500 atualizações.

5. Como ocorre o treinamento

Considere que a rede acabou de processar um dos lotes do treinamento. Explique, em sequência, como ocorre o processo de aprendizagem a partir da saída prevista até a atualização dos pesos e biases, destacando o papel da função de perda, do gradiente, da backpropagation, do otimizador e do learning rate.

1) Forward pass: a rede processa o lote e a Softmax gera as probabilidades das 4 categorias para cada imagem. 2) Função de perda: a Categorical Cross-Entropy compara essas probabilidades com as classes reais e calcula um valor de erro (perda) para o lote. 3) Backpropagation: propaga o erro da saída para as camadas anteriores e calcula o gradiente de cada peso e bias. 4) Gradiente: indica a direção e a intensidade com que cada parâmetro influencia a perda, ou seja, como alterá-lo para reduzi-la. 5) Otimizador: o Adam usa os gradientes para calcular a atualização de cada parâmetro. 6) Learning rate: 0,001 controla a escala (tamanho do passo) dessa atualização. 7) Os pesos e biases são atualizados e o processo se repete no próximo lote.

6. Taxa de aprendizagem

A configuração inicial utiliza learning_rate = 0,001. Durante alguns experimentos, dois integrantes sugerem valores muito diferentes: learning_rate = 0,1 e learning_rate = 0,00001. Explique como a taxa de aprendizagem afeta as atualizações realizadas nos pesos e biases, relacionando-a com o gradiente. Se o gradiente indica a direção e a intensidade da influência de cada parâmetro sobre a perda, qual é o papel da taxa de aprendizagem na magnitude da atualização efetivamente realizada?

O gradiente indica a direção e a intensidade da influência de cada parâmetro sobre a perda; o learning rate multiplica o gradiente e define quanto dessa informação é aplicada, ou seja, o tamanho do passo. Ele não define a direção do ajuste, apenas sua magnitude. Com 0,1, os passos tendem a ser grandes demais: o treinamento pode oscilar, ficar instável e não convergir. Com 0,00001, os passos tendem a ser muito pequenos: o treinamento avança lentamente e, nas 2500 atualizações previstas, a rede pode não aprender o suficiente. O valor 0,001 é uma escala intermediária entre estabilidade e velocidade.

7. Por que considerar uma CNN?

A primeira solução proposta utiliza uma MLP. Posteriormente, a equipe decide estudar uma CNN. Explique por que uma CNN é particularmente adequada para trabalhar com imagens, considerando sua estrutura espacial, o uso de filtros em regiões locais, a reutilização dos mesmos pesos e a geração de mapas de características.

Imagens têm estrutura espacial: pixels vizinhos são relacionados e formam bordas, texturas e formas. A CNN trabalha com a imagem em altura × largura × canais, sem transformá-la em vetor como a MLP. Seus filtros analisam regiões locais, aprendendo características locais. Os mesmos pesos de cada filtro são reutilizados em todas as posições da imagem, permitindo reconhecer o mesmo padrão em qualquer região com poucos parâmetros. Cada filtro gera um mapa de características, que indica onde aquela característica foi encontrada. Com várias camadas, a rede combina padrões simples em padrões mais complexos, o que ajuda a distinguir camiseta, calçado, bolsa e acessório.

8. Primeira camada convolucional

Considere novamente Conv2D(filters=12, kernel_size=(3,3), activation='relu') recebendo uma imagem RGB. Determine:

a) quantos filtros existem nessa camada;

12 filtros.

b) qual é a dimensão completa de cada filtro, considerando os canais da entrada;

3 × 3 × 3 (kernel 3 × 3 e 3 canais RGB).

c) quantos mapas de características serão produzidos;

12 mapas de características, um por filtro.

d) quantos pesos existem em cada filtro;

3 × 3 × 3 = 27 pesos.

e) quantos parâmetros treináveis existem em cada filtro, considerando também o bias;

27 pesos + 1 bias = 28 parâmetros.

f) quantos parâmetros treináveis possui toda a camada convolucional.

12 × 28 = 336 parâmetros treináveis.

9. Tamanho da imagem e quantidade de parâmetros

Depois do primeiro experimento, a empresa decide utilizar imagens de maior resolução: 64 × 64 × 3. Mantendo a primeira camada oculta da MLP com 32 neurônios:
a) quantos valores passarão a existir na entrada da MLP;

64 × 64 × 3 = 12288 valores.

b) quantos parâmetros treináveis passará a possuir somente essa primeira camada oculta;

(12288 × 32) + 32 = 393248 parâmetros.

c) mantendo filters=12, kernel_size=(3,3) e imagens RGB, quantos parâmetros treináveis continuará possuindo a primeira camada convolucional;

Continua com 336 parâmetros: 12 × (3 × 3 × 3 + 1) = 336.

10. MLP versus CNN

Explique por que aumentar a dimensão da imagem aumenta diretamente a quantidade de parâmetros da primeira camada densa de uma MLP, mas não aumenta a quantidade de parâmetros de uma camada convolucional.

Na MLP, a imagem vira um vetor e cada valor de entrada se conecta a todos os neurônios da primeira camada densa, cada conexão com seu próprio peso. Então, parâmetros = entradas × neurônios + biases: ao passar de 3072 para 12288 entradas, a camada vai de 98336 para 393248 parâmetros. Na CNN, os parâmetros dependem só do tamanho do kernel, do número de canais e da quantidade de filtros: 12 × (27 + 1) = 336. Como os mesmos pesos do filtro são reutilizados ao percorrer a imagem, aumentar a resolução só aumenta a quantidade de posições analisadas (e o tamanho dos mapas de características), não a quantidade de pesos.

11. Estrutura das duas abordagens

Compare MLP e CNN quanto à forma de representar a imagem, às conexões entre os neurônios e à reutilização de pesos.

Representação: na MLP a imagem é transformada em um vetor, perdendo a organização espacial; na CNN ela mantém sua estrutura espacial.

Conexões: na MLP as camadas são totalmente conectadas, cada neurônio ligado a todas as entradas; na CNN as conexões são locais, cada filtro analisa uma pequena região por vez.

Reutilização de pesos: na MLP cada conexão tem seu próprio peso, sem reutilização entre posições; na CNN os mesmos pesos do filtro são reutilizados em toda a imagem, resultando em muito menos parâmetros.